ZPDPatch: Enhancing Educational Program Repair by Constructing Targets from Submission Trajectories

Anonymous Author(s)

Abstract

We present ZPDPatch, a trajectory-supervision approach that constructs repair targets for educational program repair from execution evidence instead of from accepted solutions. Repair systems for programming exercises have markedly improved coverage by aligning correct peer programs or by conditioning language models on the failing submission. However, they all supervise repair toward the terminal program, so a correct repair often replaces most of what the learner wrote, and the effect of the repair target is reported together with the number of generated candidates, which raises coverage on its own. The goal of this paper is to construct repair targets from observed execution evidence, without a repository of correct solutions for the same problem. A key challenge is that the target cannot be varied alone, because changing it also changes the number of training examples, the distribution of failing sources, and the edit distance to the target. To address this, ZPDPatch applies a specialized algorithm that iteratively mines three finite improvement relations, matches outcome-equivalent peer targets within distance calipers, and selects a checkpoint triple by exact validation-coverage search, based on data accumulated from re-executed submissions. We evaluate it on 997 Seen and 250 problem-held-out Python inputs and on 181 student-held-out Java trajectories. Under source- and count-matched training, terminal targets repair 19.2 and 11.2 more percentage points, whereas intermediate targets produce repairs that are 10.08 and 6.83 AST-edit units smaller and retain 5.6 and 6.5 more source-token points; matched peer targets add 4.3 and 6.0 points, the Java replication reproduces both directions, and three fixed candidate draws alone contribute 15.1 points.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Applied computing** → *Education*; • **Computing methodologies** → *Natural language generation.*

Keywords

automated program repair, programming education, large language models, submission trajectories, execution-guided repair

1 Introduction

Automated program repair (APR) for programming education turns a failing submission into a program that passes the exercise tests, giving learners executable, code-specific feedback instead of a coarse verdict [Gazzola et al. 2019; Keuning et al. 2018; Monperrus 2018]. A repair system conditions a generator on the problem statement and the current program, emits one or more candidate programs, and validates them by execution; the candidate that passes the tests is returned as the repair [Le Goues et al. 2012; Weimer et al. 2009]. Because an exercise is solved by thousands of learners, a repair system can also draw on programs that other learners wrote for the same problem [Gulwani et al. 2018].

Educational repair has advanced along two lines. Reference-based systems cluster, align, or normalize correct peer programs and transfer the aligned fragments into the failing program [Gulwani et al. 2018; Hu et al. 2020; Rivers and Koedinger 2017; Wang et al. 2018]. Generation-based systems condition a language model on the failing program and on execution diagnostics, retrieved repair pairs, or an iterative conversation [Dai et al. 2026; Joshi et al. 2023; Liu et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]. Both lines have markedly improved repair coverage: in our own setting, the retrieval-based state of the art repairs 80.2% of failing Python submissions, and a fine-tuned generator repairs 72.0%.

However, both lines supervise and condition repair toward the same place: the *terminal* program, that is, the accepted solution. In this work, we define a *repair target* as the program that a repair model is trained to produce for a given failing source, and we distinguish *terminal* targets, the first accepted submission of a trajectory, from *intermediate* targets, later submissions that execution evidence shows to be improvements but not yet solutions. Which target is used is an underexplored axis, and it is consequential, because a repair learned toward a terminal target replaces most of what the learner wrote: as shown in Section 4.3, repairs produced by retrieval-based generation differ from the submitted program by 88.27 AST nodes on average. The axis is also hard to read off published numbers. In a scoped audit of eight educational repair studies (Section 2.2), none isolates the repair target, and five deploy or evaluate multiple generation opportunities without a matched single-candidate control, so a reported gain can come from extra candidates rather than from supervision.

In this paper, we present ZPDPatch, a trajectory-supervision approach that constructs repair targets from execution evidence in submission histories instead of defaulting to the accepted solution, and that does so without a repository of correct same-problem solutions. The core idea is that a learner's own trajectory, and the trajectories of learners whose attempts fail the same tests, already contain programs that are closer to the submitted code than the accepted solution is, and that training on those programs trades coverage for locality in a measurable way.

A key challenge is that the repair target cannot be varied in isolation. The mined relations are not nested and differ in size: our Python corpus yields 40,454 terminal pairs but 21,416 intermediate pairs, so replacing one target rule by another simultaneously changes the number of training examples, the distribution of failing sources, and the edit distance between source and target. Candidate breadth adds a second confound, since one additional generation opportunity raises coverage even when supervision is unchanged. Attributing an effect to target construction therefore requires holding sources, example counts, decoding, candidate budget, and selection fixed at the same time.

To address this challenge, we developed a specialized algorithm that iteratively *mines*, *matches*, and *selects* based on execution evidence accumulated from re-executed submissions. The Mine stage re-executes every submission against the exercise tests and emits three finite improvement relations over the resulting per-test outcome vectors. The Match stage replaces a learner's own intermediate target with another learner's program that has the exact same test outcomes and lies within a token-distance caliper, which fixes the source and the measured target quality while changing only provenance. The Select stage solves checkpoint choice as an exact validation-coverage search and returns a program through an execution gate that admits the unchanged submission as a fallback, so the returned program never lowers the observed pass rate and, under a finite edit budget, never exceeds it.

Experimental results on 997 Seen and 250 problem-held-out Python inputs from Project CodeNet, and on 181 student-held-out Java trajectories from a CS1 course, demonstrate a coverage–locality tradeoff under source- and count-matched training. Terminal targets repair 19.2 and 11.2 percentage points more inputs, whereas intermediate targets produce repairs that are 10.08 and 6.83 AST-edit units smaller and retain 5.6 and 6.5 more source-token points; direct retraining on the student-disjoint Java data replicates both directions (85.6% versus 50.8% repair rate, 4.8 more retained token points). Outcome-equivalent peer targets raise intermediate-target coverage by a further 4.3 and 6.0 points. A fixed-decoder contrast attributes 15.1 Seen points to three candidate draws alone, and under a maximum structural edit budget of 20 AST nodes our controller returns an accepted repair for 44.0% of inputs, compared with 13.9% for retrieval-based generation.

Contributions. We summarize our contributions as follows:

- **Underexplored problem:** We identify repair-target construction as an axis that educational repair leaves uncontrolled, show through a scoped audit of eight studies that candidate breadth is reported inconsistently, and quantify the coverage–locality tradeoff under matched training.
- **New technique:** Our core contribution is a specialized algorithm that mines three finite execution-evidence relations from trajectories, substitutes outcome-equivalent peer targets within distance calipers, solves checkpoint selection exactly, and gates candidates to guarantee observed-test non-regression and edit-budget compliance.
- **Extensive evaluation:** We evaluate ZPDPatch on two languages, two model scales, and two held-out populations with source-, count-, decoder-, and budget-matched controls, a withheld-test consistency audit, and five-fold problem cross-fitting, and we make the replication package available.

2 Preliminaries

2.1 Trajectory-Supervised Repair

The aim of educational program repair is to turn a failing submission into a program that passes the exercise tests. To do so, a repair system conditions a generator on a learner's observed submissions, generates complete programs, and validates them by execution. For a problem p, we define a submission as a pair (c, v), where c represents the complete source code and v denotes the online-judge verdict. A submission trajectory $\tau = [s_1, s_2, \ldots, s_n]$ is chronologically ordered and terminates at the first accepted submission. The problem context (d_p) holds the problem statement and the resource limits, and the test suite ($\mathcal{E}_p$) holds the available tests. Re-executing a submission yields a per-test outcome vector (o) that indicates the outcome of every test $e \in \mathcal{E}_p$, which is strictly more informative than the single verdict v.

A repair example is a pair of an input (x) and a target (y), where $x = (d_p, h)$ consists of the problem context and an observed prefix (h) of a trajectory, and y is a complete later program. A *target constructor* (T) decides which submissions remain in h and which later submission supplies y; the two need not be adjacent in the original trajectory. Given x, a repair model generates one complete program ($\hat{c}$), which is accepted only after execution.

Algorithm 1: Trajectory-Supervised Program Repair

Input: Problem context (d_p), trajectories (H), tests ($\mathcal{E}_p$), target constructor (T), candidate budget (k), edit budget (B).
Output: Repaired program ($\hat{c}$).

```
1  D ← ∅;
2  foreach τ ∈ H do
3  |  foreach failing c_i ∈ τ do
4  |  |  D ← D ∪ {((d_p, h_i), T(τ, i))} ;    // h_i: observed prefix
5  θ ← FineTune(D);
6  C ← {c} ;        // unchanged submission is feasible
7  for j ← 1 to k do
8  |  C ← C ∪ {Generate(θ, d_p, c)};
9  foreach ĉ ∈ Order(C) do
10 |  if Pass(ĉ) = 1 ∧ ted(c, ĉ) ≤ B then
11 |  |  return ĉ;
12 return arg max_{ĉ∈C} (Pass(ĉ), −ted(c, ĉ));
```

Algorithm 1 describes a generic trajectory-supervised repair procedure. It takes a problem context (d_p), trajectories (H), a test suite ($\mathcal{E}_p$), a target constructor (T), a candidate budget (k), and an edit budget (B). For now, let us set aside the last three inputs, which are the subject of this paper; existing systems instantiate T as terminal closure, fix k by implementation convenience, and leave $B = \infty$. At lines 1–5, the procedure pairs every failing submission with the target supplied by T and fine-tunes a generator on the resulting examples. At line 6, the candidate set is seeded with the unchanged submission, so returning the submission itself always remains possible. Lines 7–8 add up to k generated programs, and lines 9–11 return the first candidate that passes every test within the edit budget. For simplicity, we omit how outcomes are cached and how candidates are ordered. Finally, line 12 returns the candidate with the highest executed pass rate when no candidate qualifies.

Existing educational repair systems differ in how they fill three places of Algorithm 1. Reference-based systems replace the Generate function at line 8 with alignment against correct peer programs for the same problem [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]; language-model systems replace it with conditional

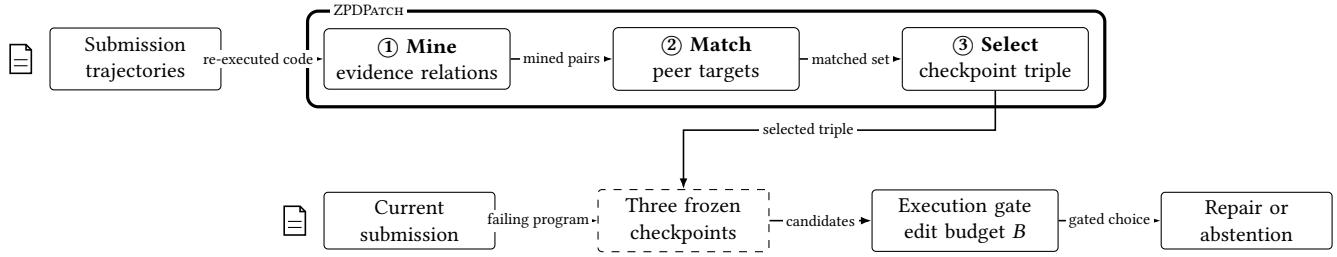


Figure 1: Overview of ZPDPatch. Offline (upper row), the three stages mine (prefix, target) pairs from re-executed trajectories, replace a target with an outcome-equivalent peer program where one exists, and select a checkpoint triple by validation coverage. At deployment (lower row), the selected checkpoints generate at most one candidate each, and the gate returns a program that never lowers the observed pass rate and never exceeds the edit budget.

generation that is optionally re-invoked with execution diagnostics [Joshi et al. 2023; Liu et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]; retrieval-based generation additionally supplies historical repair pairs written by other learners [Dai et al. 2026]. The constructor T at line 4, however, is effectively the same in all of them: each failing submission is paired with an accepted solution, and the edit budget remains infinite.

2.2 Limitation of Terminal-Target Supervision

Our key observation is that T is not treated as a design choice, even though a trajectory contains programs other than the accepted one. In this work, we define an *intermediate target* as a later submission of the same trajectory that improves on the current submission under observed execution evidence but is itself not accepted. More precisely, a later submission s_t is *reachable* from s_r when it occurs in the same learner's trajectory and either its verdict is strictly better, or the two verdicts are equal while o_t is no worse than o_r on every test and strictly better on at least one. Reachability is an empirical property of the observed snapshot; it implies neither minimality nor educational value.

Suppose a learner submits a program that fails six of ten tests, then a revision that fails two of those six, and finally an accepted program. Terminal closure pairs the first program with the accepted one and discards the revision. In this case, we classify the revision as an intermediate target of the first program: it is reachable under the evidence order above, and it stays closer to what the learner actually wrote.

Table 1: Scoped audit of candidate accounting. "1-draw" asks whether a matched single-opportunity quantity accompanies multi-opportunity coverage; "partial" denotes AVG-5 beside TOP-5. The artifact records sources and NR fields.

Study	Opportunities	Union/iterate	1-draw
PyDex/MMAPR [Zhang et al. 2022, 2024]	10/phase	yes	no
EPR benchmark [Koutcheme et al. 2024]	1/5/10	yes	yes
CREF [Yang et al. 2024]	5	yes	partial
PaR [Zhao et al. 2024]	NR + oracle	oracle	no
FastFixer [Liu et al. 2024]	1 reported	no	n/a
CEGIS repair [Orvalho et al. 2025]	1–7	yes	no
LSGen [Dai et al. 2026]	3	yes	no
REFERENT [Heo et al. 2023]	NR	NR	NR

Terminal supervision carries a measurable cost in how much of the submission survives a repair. As shown in Section 4.3, repairs returned by retrieval-based generation differ from the submitted program by 88.27 AST nodes on average and those returned by an execution-aware zero-shot baseline by 27.25; under a maximum structural edit budget of 20 AST nodes, retrieval-based generation returns an accepted repair for only 13.9% of inputs. The cost is also hard to read off published results, because the number of generated candidates is reported inconsistently. Table 1 codes eight directly related neural or LLM educational-repair studies using a rule frozen before our extended breadth results. Five deploy or primarily evaluate multiple generated opportunities. Only the EPR benchmark reports pass@1/5/10, while CREF exposes AVG-5 beside TOP-5; PyDex, counterexample-guided repair, and LSGen do not isolate extra opportunities with a matched one-candidate control [Dai et al. 2026; Koutcheme et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]. Adaptive iterations can carry new evidence, so this observation motivates explicit candidate accounting rather than invalidating those methods.

This purposive audit covers only eight studies. Its reporting contract requires same-decoder one-draw repair rate, union coverage at each deployed k, a fixed selection rule, and calls with explicit timing units. Section 4.4 supplies these quantities. Our non-adaptive candidates permit exact replay; adaptive feedback methods can change the candidate distribution and require different accounting.

One alternative is to mine every later submission as a target. However, this naive constructor changes more than the target: the mined relations are not nested and differ in size, yielding 40,454 terminal and 21,416 intermediate training pairs in our Python corpus, so example counts and the distribution of failing sources move together with the target. A second alternative is to leave T unchanged and raise the candidate budget k instead. As shown in Section 4.4, three fixed stochastic draws alone contribute 15.1 repair-rate points on Seen inputs, which exceeds most target effects reported in this paper, so an uncontrolled budget can hide or manufacture an apparent target effect.

Goal. These observations motivate a target constructor that is mined from finite execution evidence and evaluated under matched conditions. We call a constructor successful when it preserves more of the submitted program at equal coverage, or raises coverage at equal preservation, under a stated edit budget. Our work thus

tackles two challenges: (1) instantiating T from auditable per-test evidence rather than from assumed pedagogical value, and (2) separating the effect of T from the candidate budget (k), the number of training examples, and the checkpoint pool, each of which changes coverage on its own.

3 Our Approach

3.1 Overview

Figure 1 shows how ZPDPatch constructs repair targets while interacting with a generator and an execution environment. ZPDPatch operates in three stages: Mine, Match, and Select. Initially, in the Mine stage, every submission of every trajectory is re-executed against the exercise tests, and three finite relations over the resulting per-test outcome vectors emit (prefix, target) pairs: Progress mines verdict improvements and same-verdict per-test improvements, Strict requires a strictly better verdict, and Answer pairs each failure with the first accepted submission. In the Match stage, a mined target may be replaced by another learner's program that has the exact same test outcomes and lies within a token-distance caliper, which changes provenance while holding the source and the measured target quality fixed. In the Select stage, one checkpoint is fine-tuned per relation and seed, an exact search returns the triple with the best validation coverage under an optional edit budget, and inference executes the resulting candidates behind a non-regressive gate. The key novelty of ZPDPatch lies in the Mine and Match stages: the straightforward alternative of pairing every failure with the accepted solution is exactly the terminal constructor whose coverage–locality cost we measure in Section 4.2. A typical usage scenario runs the three offline stages once per corpus and then serves repairs with the three frozen checkpoints.

Algorithm 2: ZPDPatch

Input: Problem context (d_p), trajectories (H), tests ($\mathcal{E}_p$), donor pool (P), edit budget (B).
Output: Repaired program ($\hat{c}$).

/* Step 1. Mine stage */
1 $D \leftarrow \emptyset$;
2 **foreach** $\tau \in H$ **do**
3 $\quad O \leftarrow \textsc{Execute}(\tau, \mathcal{E}_p)$; // cached per-test outcomes
4 $\quad$ **foreach** *relation* $r \in \{\textsc{Progress}, \textsc{Strict}, \textsc{Answer}\}$ **do**
5 $\quad\quad D \leftarrow D \cup \textsc{Mine}(\tau, O, r)$;
/* Step 2. Match stage */
6 $D_m \leftarrow \textsc{Match}(D, P)$; // outcome-equivalent peer targets
/* Step 3. Select stage */
7 $V \leftarrow \{\textsc{FineTune}(D_m, r, s) \mid r \in \{\textsc{Progress}, \textsc{Strict}, \textsc{Answer}\}, s \in \mathit{Seeds}\}$;
8 $S \leftarrow \arg\max_{S \subseteq V, |S|=3} f_B(S)$; // exact validation search
/* Inference */
9 $C \leftarrow \{c\} \cup \{\textsc{Generate}(v, d_p, c) \mid v \in S\}$;
10 **return** $\textsc{Gate}(C, B)$;

3.2 ZPDPatch

Algorithm 2 details how ZPDPatch instantiates the constructor (T), the candidate budget (k), and the gate that Algorithm 1 leaves open. It accepts the problem context (d_p), the trajectories (H), the tests ($\mathcal{E}_p$), a donor pool (P) of other learners' submissions, and an edit budget (B); no user-supplied seed corpus, reference solution, or parameter space is required. At lines 1–5, the Mine stage re-executes each trajectory once, caches the per-test outcome vectors, and emits the relation-specific pairs. At line 6, the Match stage rewrites the target of a mined pair whenever an eligible peer target exists. At lines 7–8, the Select stage fine-tunes one checkpoint per relation and seed and searches all triples for maximum validation coverage; the selected triple (S) is then frozen. At inference, lines 9–10 generate at most one candidate per selected checkpoint and return the gated choice, so the deployed call budget equals $|S|$ and never grows with the corpus. We set $|S| = 3$ and three seeds per relation, giving nine checkpoints, by trial and error on validation coverage alone; Section 4.4 reports what the same budget buys without relation-mined targets.

3.3 Mine Stage

The goal of the Mine stage is to emit (prefix, target) pairs whose targets are justified by observed execution evidence rather than by assumed pedagogical value (lines 1–5). To do so, we group submissions by problem and learner, sort them by time, and stop at the first acceptance. After indentation, comment, and blank-line normalization we remove only consecutive duplicates, so a learner who returns to an earlier idea still contributes evidence of a distinct attempt. Complete source, verdict, and order are preserved, with no maximum history or transition count. Every state is re-executed, and cache keys bind problem, normalized code, test-suite identity, timeout, and memory limit, which prevents incompatible reuse across configurations. Runner failures receive an explicit most-severe outcome, and Mine requires a complete outcome vector before emitting a pair.

To order evidence, we assign severity 0 to AC, 1 to WA, 2 to TLE/MLE, 3 to RE, 4 to CE, and 5 to internal failures. This operational preorder mines labels; it does not claim that every WA program is semantically closer to correctness than every RE program. For equal, non-empty test keys, $o_t \preceq_{\text{sev}} o_r$ holds iff no outcome in o_t is more severe than its counterpart in o_r. Test-case progress is then the Pareto condition

$$\text{TCProg}(r, t) = \mathbf{1}[o_t \preceq_{\text{sev}} o_r \wedge o_t \neq o_r]\,. \tag{1}$$

Intuitively, TCProg fires exactly when a revision fixes at least one test without breaking another, which is invisible to the coarse verdict.

The three relations differ in which later submission they retain. Answer applies terminal closure: if s_n is accepted, every preceding non-accepted submission is independently paired with c_n, so for $\tau = [s_1, \ldots, s_9]$ the rule produces $([s_1], c_9), \ldots, ([s_8], c_9)$ and no multi-submission history. Strict keeps a retained scan: starting with $R = [s_1]$, it appends s_t only when $\text{sev}(v_t) < \text{sev}(v_r)$ for the last retained submission $s_r = R[-1]$, and emits $([r_1, \ldots, r_{k-1}], c_{r_k})$ for every $k = 2, \ldots, m$; the comparison is against the last retained state, not necessarily the immediately preceding submission. Progress uses the same scan but also appends s_t when $v_t = v_r$ and Equation (1)

holds, so it captures monotone per-test progress that an unchanged verdict hides. Formally, writing $s \prec_S t$ when $\mathrm{sev}(v(t)) < \mathrm{sev}(v(s))$,

$$s \prec_P t \iff s \prec_S t \ \vee \ (v(s) = v(t) \wedge o(t) \prec_\times o(s)),$$

where the strict product order $\prec_\times$ requires every test to be no worse and at least one to be better. Thus $\prec_S \subseteq \prec_P$ as pairwise relations, although the edge sets emitted by independent retained scans need not be nested.

For example, consider a trajectory WA → improved WA → AC. Answer connects both failures directly to the accepted program, Strict connects the first state to the accepted one because the intermediate verdict is unchanged, and Progress emits the two intervening edges. Both scans are acyclic, because each edge lexicographically decreases verdict severity and summed test severity. The guarantee concerns mined labels only: incomplete outcome vectors fail closed, and generated programs still require execution.

The severity order is a design choice, so we measured its sensitivity before running any outcome comparison: 89.5% and 90.7% of Strict and Progress labels remain valid if RE is placed below WA, and 82.6% and 84.6% remain under a binary accepted/non-accepted order. We also retrain both relations under the binary alternative, as reported in Section 4.5, rather than universalizing the canonical ranking.

3.4 Match Stage

The goal of the Match stage is to vary where a target comes from while holding what the target achieves fixed (line 6). Same-trajectory reachability is only one way to instantiate an intermediate target: another learner who failed the same tests may have written a program that is equally close to the current submission. For current code c, same-user target y_u, token edit distance d, and execution vector o, an eligible peer y_v must belong to another learner on the same problem and satisfy

$$o(y_v) = o(y_u) \ \wedge \ (\delta \le 2 \ \vee \ \delta / \max(1, d(c, y_u)) \le 0.1),$$

where $\delta = |d(c, y_v) - d(c, y_u)|$ and the disjunction applies only to the distance caliper, never to the outcome equality. Intuitively, the peer target must be indistinguishable from the same-user target in what the tests report and in how far it sits from the submitted program, so a difference in repair behavior can be attributed to provenance rather than to target quality or edit size.

Matching greedily visits scarce evidence groups first and minimizes distance discrepancy, then donor reuse and identifier, subject to a reuse cap of three; it is not a global minimum-cost matching, and we report the resulting donor distribution instead of claiming optimality. The construction is deliberately conservative: only 24.4% of mined Progress training records and 11.4% of validation records admit an eligible peer, so the matched population is a selected subset that we define explicitly rather than a corpus-wide replacement. Donors are drawn only from the same split as the recipient, which keeps validation targets out of training. Empirically, the substitution raises intermediate-target coverage by 4.3 and 6.0 points, as shown in Section 4.2.

3.5 Select Stage

The goal of the Select stage is to turn independently trained checkpoints into a small deployed portfolio and to return a program that cannot be worse than the submission (lines 7–10). All checkpoints share one role-neutral prompt containing the statement, the limits, the stored submissions with execution and edit summaries, and a request for a complete solution; no policy name or edit-size instruction is supplied, and independent QLoRA adapters minimize cross entropy on assistant target tokens only.

Training loss does not measure repair complementarity, so selection is driven by executed validation repairs. For independently trained checkpoints $V = V_P \dot\cup V_S \dot\cup V_A$, let $R_v \subseteq U$ be the validation instances repaired by v and maximize

$$f(S) = \Big|\bigcup_{v \in S} R_v\Big| \Big/ |U|.$$

Intuitively, f rewards checkpoints that repair different instances rather than the same ones, which training loss cannot distinguish. We exhaustively search all $\binom{9}{3} = 84$ triples and the 27 one-per-relation triples on one fixed-hash trajectory from each of 461 validation problems, so the reported triple is the exact maximizer rather than a heuristic choice. For $\mathcal{B} = \{5, 10, 20, 40, 80, 160\}$, the budget-indexed variant R_v^B retains only repairs whose edit distance is at most B, and we maximize either f_B or its mean over $\mathcal{B}$. Test outcomes never enter selection.

At inference, the selected checkpoints receive the same observed input and independently emit one program each; candidates never enter another checkpoint's prompt, so the portfolio is non-adaptive and exactly replayable. For a budget $B \in [0, \infty]$, the feasible set is

$$C_B = \begin{cases} \{c_i\} \cup \{c^{(j)}\}_{j=1}^{k}, & B = \infty, \\ \{c_i\} \cup \{c^{(j)} : \mathrm{ted}^\dagger(c_i, c^{(j)}) \le B\}, & B < \infty, \end{cases} \tag{2}$$

where $\mathrm{ted}^\dagger(c_i, c) = 0$ for the unchanged fallback, the Python AST tree edit distance when both changed programs parse, and $+\infty$ otherwise. A finite budget therefore rejects changed candidates when the submission does not parse, whereas unrestricted operation imposes no distance gate and can repair syntactically invalid input. The scan stops at the first accepted candidate in C_B; if none is accepted, selection is lexicographic,

$$c^* = \arg\max_{c \in C_B} \Big(\mathrm{Pass}(c), -\mathrm{ted}^\dagger(c_i, c)\Big), \tag{3}$$

which maximizes executed correctness and then minimizes change. Because the unchanged submission is always feasible, three properties follow directly from Equations (2) and (3): selection never lowers the observed pass rate, every returned change has defined edit distance at most B, and the exhaustive scan returns an eligible acceptance whenever one is emitted, so candidate order affects calls and patch identity but not fixed-set coverage. When no candidate improves the submission, the zero-distance fallback makes the system abstain instead of returning a regression. These properties constrain the controller, not the learned behavior or hidden-test correctness.

Mining costs $O(nT)$ time and $O(n)$ storage for a trajectory of n submissions and T tests, and terminal closure costs $O(n)$. Replaying the deployed portfolio sums generation and candidate execution to 13.62 seconds per input, against 24.40 for retrieval-based generation, excluding cached input execution; nine 155MB adapters occupy 1.4GB. In this manner, without a same-problem solution repository, ZPDPatch mines, matches, and selects repair targets

from execution evidence accumulated over observed trajectories, and returns a gated program whose edit size the deployer bounds.

4 Experiments

In this section, we experimentally evaluate ZPDPatch to answer the following four research questions:

- **Target construction**: How does the repair target affect repair coverage and the amount of the submitted program that survives?
- **Repair locality**: How does the gated controller compare with zero-shot and retrieval-based repair, with and without an edit budget?
- **Component efficacy**: How much coverage comes from relation-mined targets, trajectory conditioning, candidate draws, and checkpoint identity?
- **Generality**: Do the findings hold on held-out problems and learners, a smaller model, and a second language?

We implemented ZPDPatch on Qwen2.5-Coder-7B-Instruct [Hui et al. 2024], a widely used open-weight code model, training one 4-bit QLoRA adapter [Dettmers et al. 2023] per relation and seed. All experiments were conducted on a single NVIDIA RTX 5090, and every candidate was executed in a network-disabled container that runs Python and compiles Java 21.

4.1 Experimental Settings

Table 2: Corpus after whole-trajectory context filtering. "Prefix" counts raw prefixes before relation-specific filtering.

Group	Role	Problems	Tests	Traj.	Sub.	Prefix	Users
Seen	Train	492	9,446	14,027	54,481	40,454	4,012
	Validation			1,754	6,719	4,965	1,248
	Test			1,754	6,598	4,844	1,250
Unseen	Test	54	642	259	954	695	236

Benchmarks. We build the primary corpus from Project CodeNet's Python800 benchmark [Puri et al. 2021]. We retain trajectories with at least three submissions, at least one non-accepted state before the first acceptance, and a final accepted program present in the benchmark reference set; we remove post-acceptance submissions and consecutive normalized duplicates, and require every learner to appear in at least three problems and every retained problem to contain at least two trajectories. The refined corpus contains 546 problems, 21,454 trajectories, 99,432 submissions, and 10,088 tests. Before splitting, we tokenize every possible prompt–target configuration and exclude an entire trajectory when any configuration exceeds 4,096 tokens, which removes 3,660 trajectories and leaves 17,794; no later truncation of code, history, or tests is applied. We sort problems by eligible trajectory count, assign the top 90% (492 problems) to *Seen* and the remaining 54 low-resource problems to *Unseen*, and split Seen trajectories 80:10:10 within each problem (Table 2). Evaluation uses the last non-accepted submission before the recorded acceptance, leaving 997 Seen instances from 328 problems and 250 Unseen instances from all 54 held-out problems; the estimand is therefore conditional repair of the last failed state of an eventually successful trajectory. Because CodeNet identifies online-judge users rather than enrolled students, we add a course corpus: TIKTOC supplies tests for authentic CS1 Java CodeWorkout submissions [Duan et al. 2025], and the same construction, split by student, yields 1,125 trajectories from 224 students over 17 exercises, of which 181 trajectories from 33 held-out students form the Java test set.

Baselines. We compare ZPDPatch with three baselines and one naive variant. *Zero-shot* is an execution-aware iterative baseline that uses the same base model and prompt without fine-tuning and, after a failure, appends the verdict and the number of passed tests before its next call. *Answer-3Seed* is the naive terminal constructor of Section 2.2: three adapters trained on the 40,454 terminal pairs with identical model, hyperparameters, decoder, and call budget, varying only the seed. *Answer-9Choose3* is the strictest comparison, because it trains nine terminal checkpoints and receives the same exhaustive 84-triple validation search as the mixed pool, matching prompts, decoding, pool size, and selection opportunity. *LSGen* is a retrieval-based solution generator [Dai et al. 2026]; we preserve its repair-pair filtering, edit-vector retrieval, diff-guided generation, and re-retrieval, and replay the same edit gate. Since LSGen retains other learners' Seen-Train solutions for the target problem, Unseen evaluation would expose same-problem correct programs, so we omit it there rather than supply unavailable references.

Table 3: Source-matched target control. Δ is Progress minus Answer in percentage points; locality is conditional on joint repair. Intervals cluster by problem for Python and by student for Java, and Java lexical units are language-neutral rather than Python AST nodes.

Split	Repair rate (%)		
	Progress	Answer	ΔRR [95% CI]
Seen	34.0	**53.2**	−19.2 [−22.96, −15.54]
Unseen	65.6	**76.8**	−11.2 [−17.37, −4.88]
Java students	50.8	**85.6**	−34.8 [−44.91, −25.47]
Split	**Locality on joint repairs**		
	Joint	Δ token ret.	Δ line retention
Seen	299	**5.6** [4.29, 6.94]	**8.9** [6.64, 11.16]
Unseen	154	**6.5** [4.26, 8.95]	**10.4** [6.95, 14.25]
Java students	90	**4.8** [2.79, 7.19]	**5.1** [2.92, 7.25]

Other settings. All approaches share the base model, public tests, a 2.5-second per-test timeout, a 2GB memory limit, at most three candidate generations, and a 4,096-token generation cap. Seeds 2027–2029 are repeated for every target, and Answer-9Choose3 adds seeds 2030–2035 without changing data or decoding. Pass Rate (PR) averages the test pass fraction of the returned program, Repair Rate (RR) averages whether it passes every test, and Improvement Rate (IR) averages whether it improves on the submission; $\mathrm{TED}_{B\to F}$ measures AST distance from the submitted to the returned program on parseable repairs [Pawlik and Augsten 2016; Zhang and Shasha 1989], and retention measures the fraction of submitted tokens or non-empty lines that survive. We report Wilson intervals [Wilson 1927], exact McNemar tests [McNemar 1947], and Holm correction within comparison families [Holm 1979]; primary paired intervals resample problems 10,000 times with recorded seeds, and Java reports both student- and exercise-cluster intervals [Efron 1979]. Endpoints, folds, selectors, budgets, and bootstrap draws were fixed

Table 4: Main repair results. PR, RR, and IR are percentages; TED is computed on successfully repaired, parseable programs. Bold marks the best value per column, and "–" marks the configuration that is not evaluated on Unseen because it would expose same-problem solutions.

Method	Seen (997 instances)					Unseen (250 instances)				
	PR	RR	IR	$\text{TED}_{B\to F}$	$\text{TED}_{F\to O}$	PR	RR	IR	$\text{TED}_{B\to F}$	$\text{TED}_{F\to O}$
Zero-shot	76.3	30.7	43.7	27.25	27.67	75.3	62.0	68.8	17.46	15.80
LSGen	88.0	**80.2**	**82.4**	88.27	83.23	–	–	–	–	–
Answer-3Seed (history)	87.0	61.7	69.1	24.52	24.82	84.8	73.2	79.2	13.38	13.86
Answer-1 (current)	76.6	58.1	66.1	30.27	28.54	80.0	69.6	78.8	15.30	15.18
Answer-3Seed (current)	**90.2**	72.0	78.6	37.94	35.51	**89.3**	**81.2**	**88.0**	16.01	16.58
Answer-9Choose3	85.9	59.8	68.0	25.46	24.01	85.7	74.8	80.4	15.19	13.75
ZPDPatch	87.4	61.5	68.9	**21.05**	**21.64**	84.9	74.8	79.6	**11.40**	**11.96**

before test generation, and final-test outcomes never enter training or selection, although no protocol was externally preregistered. OpenAI Codex was used interactively to propose control and validation-selection designs, implement dataset builders, orchestrate experiments, develop statistical-analysis scripts, and assist manuscript drafting. The authors selected the final design and acceptance criteria, ran jobs on author-controlled infrastructure, reviewed code changes, checked reported values against saved machine-readable outputs, and every cited work was checked against a retrievable source; AI-produced text is not empirical evidence.

4.2 RQ1: Target Construction

Table 3 shows that the repair target moves coverage and locality in opposite directions. The paired-target control inner-joins intermediate and terminal examples by problem, learner, submission identifier, and exact current code, discards pairs with the same target, and serializes only the shared current code, so 7,389 training and 931 validation pairs fix the sources, counts, seeds, prompts, selectors, and test inputs; only the target differs. Under this control, intermediate targets lose 19.2 Seen and 11.2 Unseen RR points, both clustered intervals exclude zero, and exact McNemar tests give $p = 7.85 \times 10^{-34}$ and $p = 6.17 \times 10^{-5}$. The same control confirms the opposite direction for locality: on 278 and 134 jointly repaired parseable instances, terminal patches are 10.08 and 6.83 AST-edit units larger, and across 299 and 154 joint repairs intermediate targets retain 5.6 and 6.5 more token points and 8.9 and 10.4 more line points, with every clustered interval excluding zero. Exceptionally, 21 Seen and 20 Unseen joint repairs start from syntactically invalid source code, so both returned repairs parse but their AST distance is undefined; those instances contribute to coverage and lexical retention only.

Provenance is a second lever on the same stage. Replacing each same-user intermediate target with an outcome-equivalent peer target raises coverage from 50.4% to 54.7% on Seen and from 78.4% to 84.4% on Unseen, that is, by 4.3 points [−7.71, −0.84] and 6.0 points [−11.79, −0.41] in the paired same-minus-peer direction, with exact McNemar $p = 0.005$ and $p = 0.032$. Because current source, exact target outcomes, distance calipers, example counts, prompts, seeds, and selection are all controlled, this contrast identifies the matched peer-selection rule rather than user identity itself as the stronger construction.

Direct retraining on the Java corpus replicates the tradeoff outside CodeNet and without student overlap. On 181 student-held-out trajectories, terminal targets repair 155 trajectories and intermediate targets 92, a difference of 34.8 points with student-cluster interval [−44.91, −25.47] and exercise-cluster interval [−47.31, −23.79]; of 90 joint repairs, intermediate targets retain 4.8 more lexical-token points and 5.1 more line points. These results indicate that terminal targets maximize coverage while intermediate targets maximize source preservation, and that the direction of both effects survives a change of language, population, and course setting.

4.3 RQ2: Repair Locality and Edit Budget

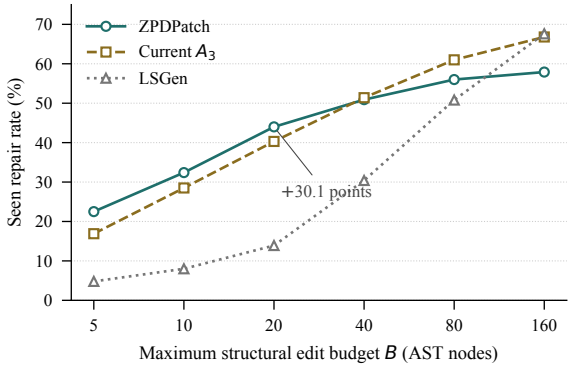


Figure 2: Seen repair rate under the same AST-edit deployment gate. ZPDPatch uses the validation-frozen budget-indexed portfolio, current-only A_3 the three recommended checkpoints, and LSGen always generates three candidates.

Table 4 shows that the execution-gated portfolio repairs 613 of 997 Seen programs (61.5%) against 306 (30.7%) for Zero-shot, with Wilson intervals [58.4, 64.5] and [27.9, 33.6]. The paired gain of 30.8 points holds under problem-cluster bootstrap [26.4, 35.1] and remains 26.1 points [21.2, 30.9] when problems are weighted equally; PR rises by 11.07 points [9.16, 13.02] and IR by 25.2 points [20.8, 29.4]. The two systems are complementary rather than nested:

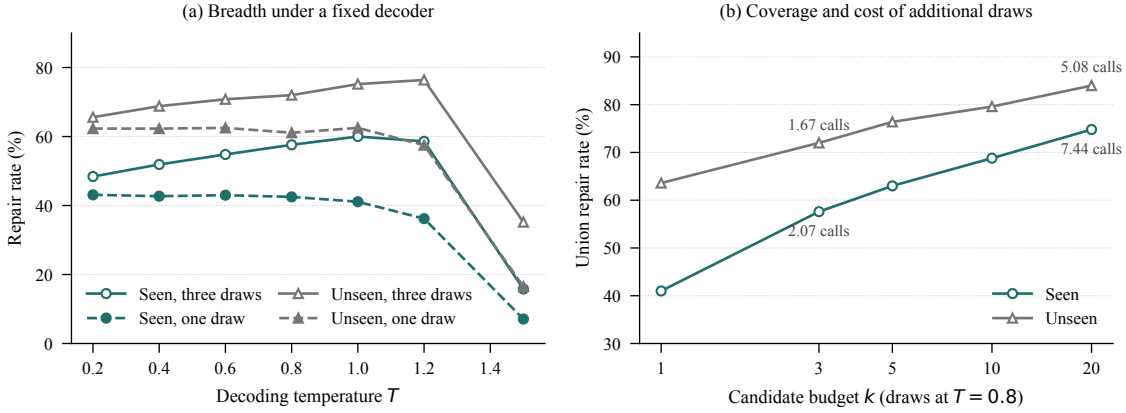


Figure 3: Candidate breadth under a fixed stochastic decoder. (a) One-draw and three-draw union repair rate across every frozen temperature; (b) union repair rate as the candidate budget grows at $T = 0.8$, annotated with mean early-stop calls.

342 instances are repaired only by ZPDPatch and 35 only by Zero-shot (exact McNemar $p < 10^{-63}$). Notably, the strongest learned configuration in this stack is the current-only terminal triple at 72.0% Seen and 81.2% Unseen, which we therefore report as the unrestricted deployment default.

Interestingly, retrieval-based LSGen achieves the highest unrestricted coverage of all systems, with 88.0% PR, 80.2% RR, and 82.4% IR; it repairs 267 instances that ZPDPatch misses while ZPDPatch repairs 80 that it misses ($p < 10^{-23}$), and the clustered RR difference is −18.8 points [−23.0, −14.5]. The difference is paid in patch size. Mean $\mathrm{TED}_{B \to F}$ is 21.05 for ZPDPatch against 27.25 for Zero-shot and 88.27 for LSGen, and on jointly repaired instances ZPDPatch reduces distance by 8.46 units [5.05, 12.55] and 43.34 units [38.47, 48.44], so the comparison is not confounded by different success sets.

Figure 2 operationalizes that tradeoff under a deployment gate. At a maximum structural edit budget of 20 AST nodes, the gate returns an accepted repair for 44.0% of inputs with ZPDPatch against 13.9% with always-three LSGen, a problem-clustered difference of 30.09 points [26.45, 33.82]; across the six predeclared budgets the mean difference is 14.68 points [12.03, 17.43]. LSGen crosses over only at the broadest budget, where it leads by 9.7 points. The relation-level ordering agrees: giving each adapter a single candidate on the same 997 instances, terminal supervision reaches the highest RR but the largest mean patch (TED 20.22), while intermediate supervision gives up 5.2 RR points [2.5, 7.9] for the smallest patch (TED 15.85); on 350 joint repairs the paired difference is 1.84 units [0.50, 3.26], although the median gap across 341 compact joint repairs is zero, so the effect lives in the tail. These results indicate that retrieval wins unrestricted coverage whereas the gated portfolio covers more inputs whenever large structural replacement is disallowed. We treat edit distance only as an explicit change-cost axis: the benchmark supplies no evidence that a particular budget is an educational requirement.

4.4 RQ3: Component Efficacy and Candidate Breadth

For fixed candidates, let Z_{ij} indicate that draw j repairs instance i and $M_i = \sum_{j=1}^{k} Z_{ij}$. The realized contribution of breadth alone is exactly

$$\widehat{\Delta}_{\text{breadth}} = \frac{1}{N} \sum_{i} \left[\mathbf{1}(M_i > 0) - \frac{M_i}{k} \right] \geq 0, \tag{4}$$

which is positive exactly when success differs across draws and therefore cannot be attributed to temperature. Figure 3 applies Equation (4) under a fixed stochastic decoder. Replacing greedy decoding by one stochastic draw costs 7.6 points [−9.81, −5.41], whereas execution-selecting three of those identical fixed draws contributes 15.1 points [13.48, 16.76]. Through $T = 1.0$ the one-draw rate stays within 41.1–43.1% on Seen, the three-draw union reaches 60.0/75.2%, and at $T = 1.5$ the decoder collapses to 15.8/35.2%. Extending the same candidate stream raises coverage monotonically to 74.8/84.0% at 20 draws, at 7.44/5.08 mean calls. Fine-tuning contributes a second, larger effect: the matched base-model three-draw union reaches only 30.4/65.6%, so adaptation adds 27.2 points [22.99, 31.41] on Seen and 6.4 points [1.19, 11.34] on Unseen.

Checkpoint identity, by contrast, adds no conclusive third effect. At matched temperature, replacing three draws of one checkpoint by one draw from each of three checkpoints changes RR by −0.1 points [−2.00, 1.80] on Seen and 2.4 points [−0.41, 5.31] on Unseen. A current-only four-cell comparison on a single inference stack agrees: three greedy checkpoints gain 9.8 [7.85, 11.92] and 10.8 [7.23, 14.64] points over one, but under the matched stochastic decoder distinct checkpoints add only 2.1 points [0.00, 4.16] and 1.2 points [−2.93, 5.93]. Absolute greedy-three coverage is 62.9/78.0% on that stack against 72.0/81.2% on the historical one, so stack sensitivity is material and no backend-independent deployment advantage follows.

Enlarging the pool or mixing relations does not raise coverage either. Growing the terminal pool from three to nine checkpoints changes RR by −1.91 points [−4.30, 0.32]; the primary five-fold cross-fitted mixed-minus-terminal contrast is +2.2 points [−0.19,

4.63] and the non-cross-fitted estimate +1.71 [−0.79, 4.29]. Across six budgets the mixed pool improves mean Seen RR by 1.22 points [0.02, 2.42], but the Unseen interval includes zero, the 1.5B replication reverses sign, and the user-held-out Java interval includes zero, so we reject a general mixed-target coverage claim and retain source preservation as the supported specialization: on 532 joint repairs the mixed pool preserves 2.57 more token points [1.56, 3.64] and 3.33 more line points [1.89, 4.83]. Trajectory conditioning behaves the same way. Matched retraining that removes only the earlier submissions yields no stable serialization effect, and none of the four RR contrasts survives Holm correction (minimum adjusted $p = .313$); at the portfolio level, removing history instead improves frozen mixed and terminal RR by 8.8/11.1 Seen points and 7.6/6.0 Unseen points, with all intervals excluding zero. These results indicate that candidate breadth and fine-tuning account for most of the coverage, whereas checkpoint identity, pool size, and serialized history do not.

4.5 RQ4: Generality and Replication

On the 250 problem-held-out Unseen instances, ZPDPatch reaches 74.8% RR against 62.0% for Zero-shot, a paired gain of 12.8 points [6.83, 18.75] with 45 versus 13 exclusive repairs ($p = 3.01 \times 10^{-5}$), and its patches are 6.41 AST-edit units smaller on 122 joint parseable repairs [3.43, 9.89]. Because trajectory availability rather than randomized difficulty defines the split, the higher absolute rate does not mean that unseen problems are easier: a pre-repair audit that matches all 54 Unseen problems to distinct Seen problems on six outcome-free difficulty covariates turns the raw gap into −4.6 problem-balanced points [−14.18, 5.30], which is why we rely on within-split paired contrasts.

A withheld-test audit separates candidate selection from success assessment. It splits the tests of each problem by fixed hashes, recomputes prompts and selection from the observed side alone, and scores the returned repairs on the hidden side. Under this protocol, 189 of 192 ZPDPatch repairs and 184 of 186 matched repairs also pass every hidden test, giving joint RR 75.6% versus 73.6% with 13 versus eight exclusive repairs ($p = .383$) and confirmation rates of 98.4% and 98.9%; repeating the procedure on all 328 Seen problems with a distinct seed yields 97.8% and 97.9% confirmation. Both slices come from one authored suite, so they bound slice overfitting rather than establishing semantic correctness. The conventional endpoint is also not load-bearing: across 2,770 Seen and 674 Unseen executable prefixes of eventually successful trajectories, the deployed configuration reaches 69.6% and 80.1% RR, and the early, middle, and last thirds reach 70.0%, 67.6%, and 71.4% on Seen and 82.9%, 77.3%, and 81.8% on Unseen.

Finally, the portfolio findings are stable under scale and protocol perturbations, while the mixed-pool advantage is not. At 1.5B parameters three checkpoints still gain 13.4 points over one [11.16, 15.92], but mixed-minus-terminal is −0.8 [−2.62, 1.07]; on Java the same contrast is positive on four held-out exercises (3.9 points [2.99, 5.11]) and inconclusive on held-out students (0.6 [−1.20, 2.59]); retraining both relations under the binary verdict order changes intermediate-target RR by 2.3 and −2.8 points. Exact matches between a returned repair and a training-time accepted program occur in 0.6% and 1.3% of selections, which does not exclude non-exact memorization. These results indicate that the target-construction tradeoff and the controller guarantees transfer across problems, learners, scales, and languages, whereas a general mixed-pool coverage advantage does not.

4.6 Threats to Validity

(1) *Construct.* Test passing is not semantic equivalence, and edit distance and retention measure change rather than pedagogy; we therefore report them as change costs only, and the withheld-test audit confirms 98.4% and 98.9% of repairs on untouched tests. (2) *Internal.* The paired controls fix sources and counts, and peer matching additionally fixes target outcomes within token-distance calipers, but not all structural or stylistic properties; no protocol was externally preregistered, and the artifact records every declared amendment. (3) *External.* Evidence covers one Python benchmark and model family, one smaller model, and two Java splits; Seen permits problem and user overlap, audits cannot eliminate memorization [Allamanis 2019], all-prefix evaluation includes only eventually successful trajectories, and CodeNet is not a classroom cohort, so learner utility remains unmeasured.

5 Related Work

Repairing student programs. Our work shares a common goal with systems that turn a failing student submission into a program that passes the exercise tests [Dai et al. 2026; Gulwani et al. 2018; Hu et al. 2020; Joshi et al. 2023; Liu et al. 2024; Wang et al. 2018; Zhang et al. 2024]. Reference-based systems cluster and align correct peer programs before extracting a repair, search and align reference programs, or normalize refactorings before comparison [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]; embeddings, solution-space models, and visualization propagate annotations or expose recurring strategies, while multi-function and diversified methods broaden the target beyond a single correction [Choi et al. 2021; Choi and Lee 2025; Glassman et al. 2015; Heo et al. 2023; Piech et al. 2015b; Rivers and Koedinger 2017; Yi et al. 2017]. Generation-based systems instead condition a language model on the failing program, optionally with execution diagnostics, retrieved repair pairs, peer programs, or syntax localization [Joshi et al. 2023; Liu et al. 2024; Orvalho et al. 2025; Phung et al. 2023; Yang et al. 2024; Zhang et al. 2022, 2024; Zhao et al. 2024]. Unlike ours, all of them supervise repair toward an accepted solution, and most additionally require correct programs for the same problem. The closest system is LSGen, which retrieves historical repair pairs by edit information, generates, and may retrieve again [Dai et al. 2026]. The key difference is that LSGen draws its guidance from other learners' accepted solutions for the target problem, which do not exist for a genuinely new assignment, whereas ZPDPatch mines targets from execution evidence inside observed trajectories. Since our stages change only the target and the gate, they can be combined with any of these generators.

Learning from submission histories. Our work belongs to a line of research that reads interaction histories rather than a single snapshot. Knowledge tracing estimates latent mastery from such histories [Corbett and Anderson 1994; Ghosh et al. 2020; Piech et al. 2015a], feedback research emphasizes actionable information beyond correctness [Hattie and Timperley 2007; Keuning et al.

2018; Narciss 2008; Shute 2008], and tutoring systems, iSnap, and language-model hints address help seeking and next-step granularity [Aleven and Koedinger 2000; Heickal and Lan 2024; Price et al. 2017; Roest et al. 2024; Xiao et al. 2024]. The Zone of Proximal Development motivates support between independent and assisted performance [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002; Schulze Bernd and Chounta 2024]. In contrast, ZPDPatch reads a history only to order code states by observed execution evidence: it infers no mastery, predicts no learning, and its reachability relation is narrower than the construct that motivates the name. The two readings are compatible, since a mastery estimate decides when to intervene while our stages decide what to repair toward.

Execution-guided repair and candidate breadth. Our controller is rooted in generate-and-validate repair, which exposed both the benefit of broad search and the risk of overfitting to tests [Le Goues et al. 2012; Qi et al. 2014; Smith et al. 2015; Weimer et al. 2009], and in neural repair that learns repository transformations or context-aware translation [Jiang et al. 2023; Lutellier et al. 2020; Tufano et al. 2019; Xia and Zhang 2022; Yasunaga and Liang 2021]. Execution-aware methods supply signals independent of model confidence: SelfAPR learns from test diagnostics, FixEval makes execution the evaluation criterion, CREF carries diagnostics through a repair conversation, and counterexample-guided repair combines generation, fault localization, and MaxSAT [Haque et al. 2023; Orvalho et al. 2025; Yang et al. 2024; Ye et al. 2022]. Unlike these, we use execution twice: once to define the supervision relation offline, and once to gate candidates at deployment. Repeated sampling is an established inference-scaling mechanism, because pass@k rises when independent draws cover different successful programs [Chen et al. 2021]; we do not claim that effect as novel but measure its exact contribution inside the controller, which also separates our portfolio from deep ensembles and model soups that aggregate probabilities or weights instead of executing complete programs [Lakshminarayanan et al. 2017; Wortsman et al. 2022]. These mechanisms are orthogonal to target construction, so they can be combined with it.

6 Conclusion

Recent educational repair systems have substantially improved repair coverage, yet they all supervise repair toward an accepted solution and leave the choice of repair target uncontrolled. This paper constructs repair targets from execution evidence in submission trajectories, without a repository of correct solutions for the same problem. Our approach, ZPDPatch, iteratively mines finite improvement relations, matches outcome-equivalent peer targets, and selects a checkpoint triple whose candidates are returned through an execution gate. Under source- and count-matched training on Python and on student-disjoint Java, terminal targets increase repair coverage while intermediate targets preserve substantially more of the submitted program, and matched peer targets improve coverage further. The controller guarantees observed-test non-regression and compliance with a finite edit budget.

7 Data and Artifact Availability

Our anonymized replication package records the runtime environment, a hash-bound manifest, and an `ARTIFACT.md` claim–evidence map that links each table and figure to the analysis output behind it. A public archival identifier is not yet assigned.

References

Vincent Aleven and Kenneth R Koedinger. 2000. Limitations of student control: Do students know when they need help?. In *International conference on intelligent tutoring systems*. Springer, 292–303.

Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 143–153. doi:10.1145/3359591.3359735

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374

Dongwook Choi, Jinseok Heo, and Eunseok Lee. 2021. Automated Feedback Generation for Multiple Function Programs. In *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 582–583.

Dongwook Choi and Eunseok Lee. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 230–241.

Irene-Angelica Chounta, Patricia Albacete, Pamela Jordan, Sandra Katz, and Bruce M McLaren. 2017. The "Grey Area": A computational approach to model the Zone of Proximal Development. In *European conference on technology enhanced learning*. Springer, 3–16.

Michael Cole, Vera John-Steiner, Sylvia Scribner, and Ellen Souberman. 1978. Mind in society. *Mind in society the development of higher psychological processes. Cambridge, MA: Harvard University Press* (1978).

Albert T Corbett and John R Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction* 4, 4 (1994), 253–278.

Zhenlong Dai, Zhuoluo Zhao, Hengning Wang, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. *arXiv preprint arXiv:2601.08545* (2026).

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.

Zhangqi Duan, Nigel Fernandez, Alexander Hicks, and Andrew Lan. 2025. Test Case-Informed Knowledge Tracing for Open-ended Coding Tasks. In *Proceedings of the 15th International Learning Analytics and Knowledge Conference*. 235–250. doi:10.1145/3706468.3706500

Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. doi:10.1214/aos/1176344552

Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic Software Repair: A Survey. *IEEE Transactions on Software Engineering* 45, 1 (2019), 34–67. doi:10.1109/TSE.2017.2755013

Aritra Ghosh, Neil Heffernan, and Andrew S Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2330–2339.

Elena L. Glassman, Jeremy Scott, Rishabh Singh, Philip J. Guo, and Robert C. Miller. 2015. OverCode: Visualizing Variation in Student Solutions to Programming Problems at Scale. *ACM Transactions on Computer-Human Interaction* 22, 2, Article 7 (2015), 35 pages. doi:10.1145/2699751

Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480.

Md Mahim Anjum Haque, Wasi Uddin Ahmad, Ismini Lourentzou, and Chris Brown. 2023. Fixeval: Execution-based evaluation of program fixes for programming problems. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 11–18.

John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487

Hasnain Heickal and Andrew Lan. 2024. Generating feedback-ladders for logical errors in programming using large language models. *arXiv preprint arXiv:2405.00302* (2024).

Jinseok Heo, Hohyeon Jeong, Dongwook Choi, and Eunseok Lee. 2023. Referent: Transformer-based feedback generation using assignment information for programming course. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 101–106.

Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. doi:10.2307/4615733

Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2020. Re-factoring based program repair applied to programming assignments. In

Proceedings-2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019. IEEE, 388–398.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *Proceedings of the 45th International Conference on Software Engineering*. 1430–1442. doi:10.1109/ICSE48619.2023.00125

Harshit Joshi, José Cambronero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140.

Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education (TOCE)* 19, 1 (2018), 1–43.

Charles Koutcheme, Nicola Dainese, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. 2024. Benchmarking Educational Program Repair. *arXiv preprint arXiv:2405.05347* (2024). doi:10.48550/arXiv.2405.05347

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles. In *Advances in Neural Information Processing Systems*, Vol. 30.

Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2012), 54–72. doi:10.1109/TSE.2011.104

Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.

Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: Combining Context-Aware Neural Translation Models Using Ensemble for Program Repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 101–114. doi:10.1145/3395363.3397369

Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996

Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *Comput. Surveys* 51, 1, Article 17 (2018), 24 pages. doi:10.1145/3105906

Tom Murray and Ivon Arroyo. 2002. Toward measuring and maintaining the zone of proximal development in adaptive instructional systems. In *International conference on intelligent tutoring systems*. Springer, 749–758.

Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. In *Handbook of research on educational communications and technology*. Routledge, 125–143.

Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.

Mateusz Pawlik and Nikolaus Augsten. 2016. Tree edit distance: Robust and memory-efficient. *Information Systems* 56 (2016), 157–173. doi:10.1016/j.is.2015.08.004

Tung Phung, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating high-precision feedback for programming syntax errors using large language models. *arXiv preprint arXiv:2302.04662* (2023).

Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. 2015a. Deep knowledge tracing. *Advances in neural information processing systems* 28 (2015).

Chris Piech, Jonathan Huang, Andy Nguyen, Mike Phulsuksombati, Mehran Sahami, and Leonidas Guibas. 2015b. Learning Program Embeddings to Propagate Feedback on Student Code. In *Proceedings of the 32nd International Conference on Machine Learning*. 1093–1102.

Thomas W Price, Yihuan Dong, and Dragan Lipovac. 2017. iSnap: towards intelligent tutoring in novice programming environments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*. 483–488.

Ruchir Puri, David S Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladmir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, et al. 2021. Project codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. *arXiv preprint arXiv:2105.12655* 1035 (2021).

Yuhua Qi, Xiaoguang Mao, Yan Lei, Ziying Dai, and Chengsong Wang. 2014. The Strength of Random Search on Automated Program Repair. In *Proceedings of the 36th International Conference on Software Engineering*. 254–265. doi:10.1145/2568225.2568254

Kelly Rivers and Kenneth R. Koedinger. 2017. Data-Driven Hint Generation in Vast Solution Spaces: A Self-Improving Python Programming Tutor. *International Journal of Artificial Intelligence in Education* 27, 1 (2017), 37–64. doi:10.1007/s40593-015-0070-z

Lianne Roest, Hieke Keuning, and Johan Jeuring. 2024. Next-step hint generation for introductory programming using large language models. In *Proceedings of the 26th Australasian Computing Education Conference*. 144–153.

Nadine Schulze Bernd and Irene-Angelica Chounta. 2024. Beyond the grey area: exploring the effectiveness of scaffolding as a learning measure. In *International Conference on Artificial Intelligence in Education*. Springer, 365–378.

Valerie J. Shute. 2008. Focus on Formative Feedback. *Review of Educational Research* 78, 1 (2008), 153–189. doi:10.3102/0034654307313795

Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*. 532–543. doi:10.1145/2786805.2786825

Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation. *ACM Transactions on Software Engineering and Methodology* 28, 4, Article 19 (2019), 29 pages. doi:10.1145/3340544

Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*. 481–495.

Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. 2009. Automatically Finding Patches Using Genetic Programming. In *Proceedings of the 31st International Conference on Software Engineering*. 364–374. doi:10.1109/ICSE.2009.5070536

Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953

Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. 2022. Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy without Increasing Inference Time. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 23965–23998.

Chunqiu Steven Xia and Lingming Zhang. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971. doi:10.1145/3540250.3549101

Ruiwei Xiao, Xinying Hou, and John Stamper. 2024. Exploring how multiple levels of GPT-generated programming hints support or disappoint novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–10.

Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 882–894.

Michihiro Yasunaga and Percy Liang. 2021. Break-it-fix-it: Unsupervised learning for program repair. In *International conference on machine learning*. PMLR, 11941–11952.

He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

Jooyong Yi, Umair Z Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 740–751.

Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876* (2022).

Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 1100–1124.

Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. doi:10.1137/0218082

Qianhui Zhao, Fang Liu, Li Zhang, Yang Liu, Zhen Yan, Zhenghao Chen, Yufei Zhou, Jing Jiang, and Ge Li. 2024. Peer-aided repairer: Empowering large language models to repair advanced student assignments. *arXiv preprint arXiv:2404.01754* (2024). doi:10.48550/arXiv.2404.01754